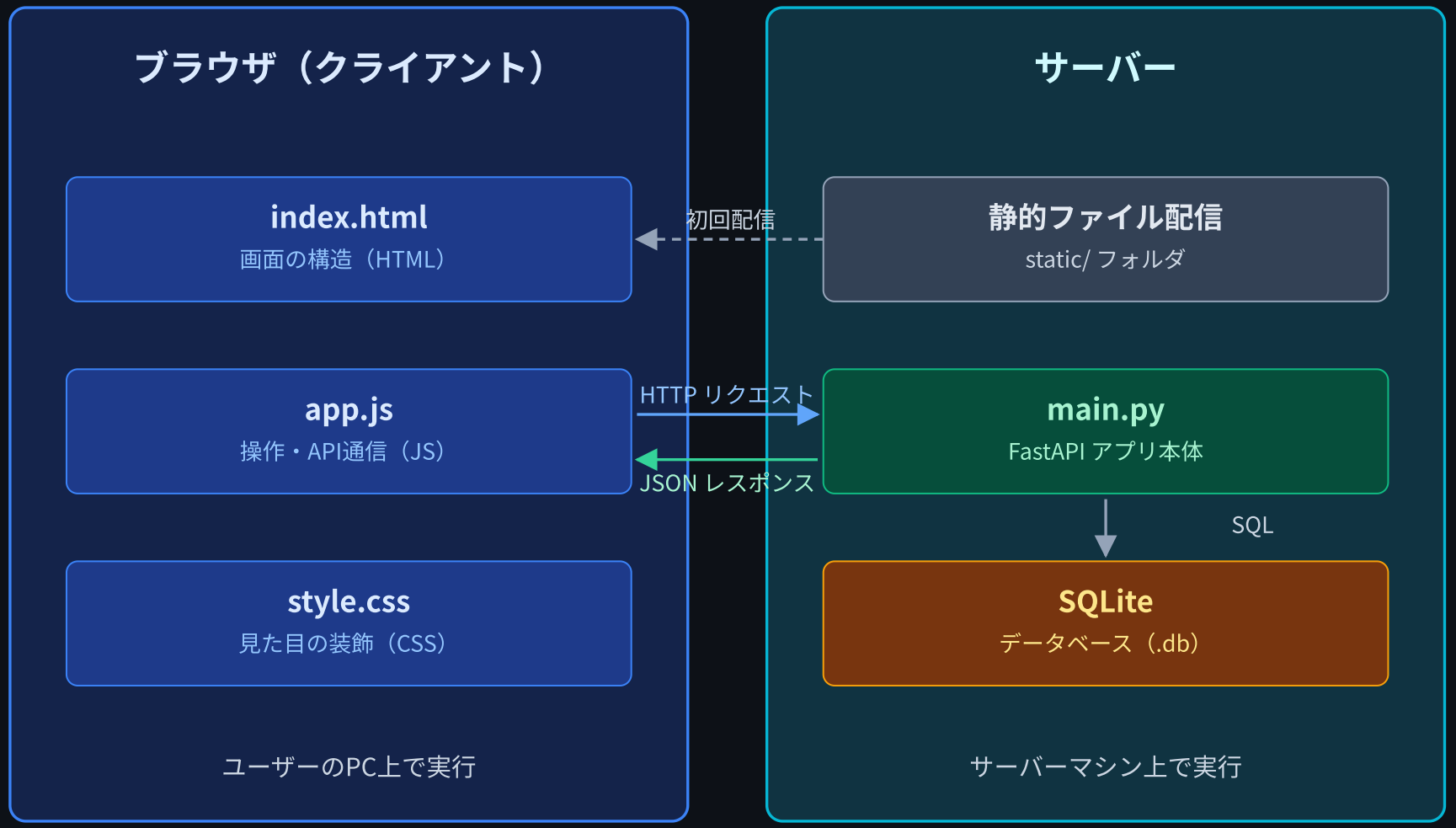
第5回: Python & FastAPI入門 — サーバーサイドを理解する

Webアプリケーション基礎 2026

今日のゴール

サーバー上でAPIが動くことを体験する


ブラウザ（クライアント）
index.html
画面の構造（HTML）
app.js
操作・API通信（JS）
style.css
見た目の装飾（CSS）
ユーザーのPC上で実行
初回配信
HTTP リクエスト
JSON レスポンス
サーバー
静的ファイル配信
static/ フォルダ
main.py
FastAPI アプリ本体
SQL
SQLite
データベース（.db）
サーバーマシン上で実行

今日の流れ

前半

- なぜバックエンドが必要か
- Python基礎 — 変数・型・リスト・辞書
- Python基礎 — 関数・条件分岐・ループ

後半

- FastAPIとは — Hello World
- ルーティングとエンドポイント
- JSONとデータ交換

セクション1

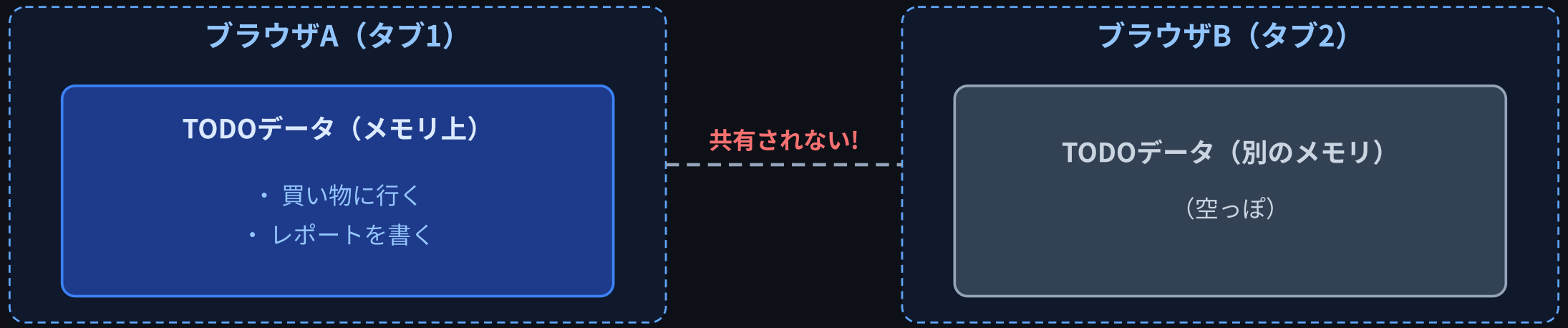
なぜバックエンドが必要か

前回までの振り返り

- 第1回〜第4回で **フロントエンド** のTODOアプリを作った
- HTML / CSS / JavaScript で見た目と動きを実装
- **全てのコードがブラウザ上で動いている**

ここで1つ問題がある...

フロントエンドだけの限界



バックエンドが必要な3つの理由

1. データの永続化

- ブラウザを閉じてもデータが消えない
- サーバーのデータベースに保存する

2. 複数ユーザー対応

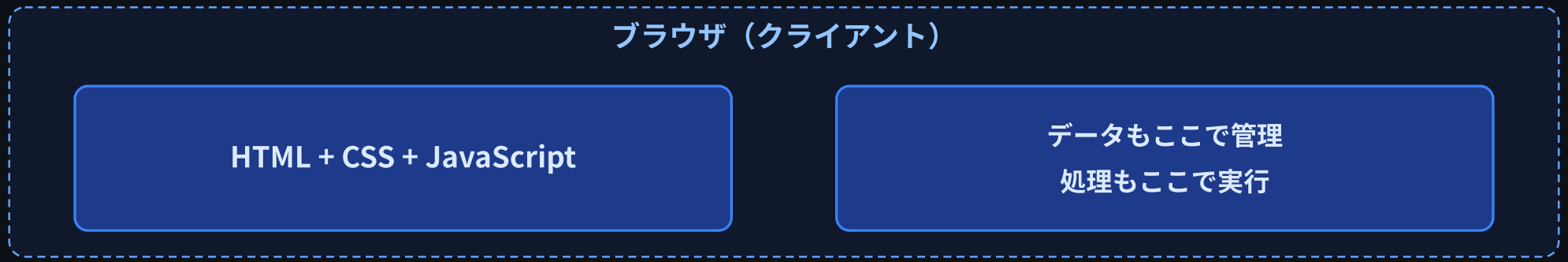
- 異なるブラウザ・デバイスから同じデータにアクセス
- チームでTODOを共有できる

3. セキュリティ

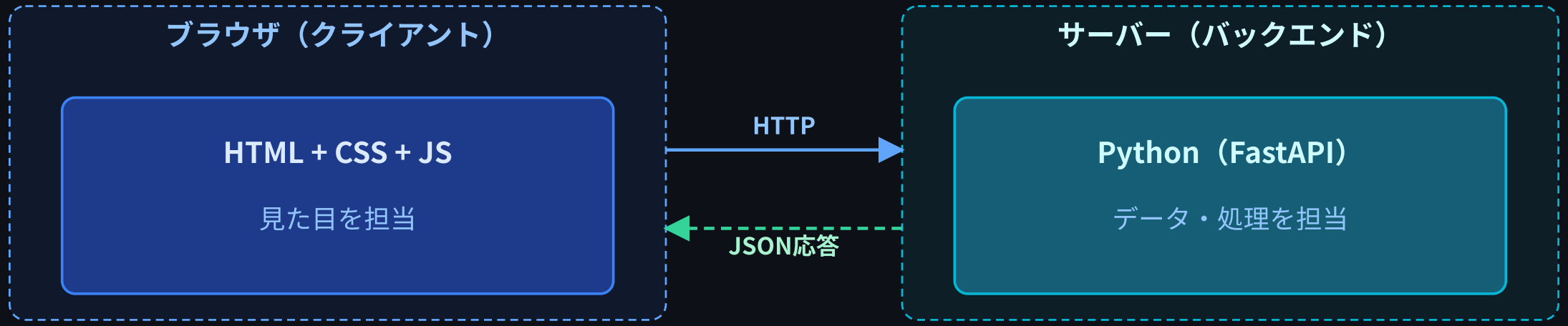
- 重要な処理はサーバー側で行う
- ユーザーの入力を検証する
- APIキーなどの秘密情報を守る

コードの実行場所が変わる

【これまで（フロントエンドのみ）】



【これから（フロントエンド + バックエンド）】



クライアントとサーバーの役割分担

	クライアント (ブラウザ)	サーバー (バックエンド)
言語	HTML / CSS / JavaScript	Python (FastAPI)
役割	画面表示・ユーザー操作	データ管理・ビジネスロジック
実行場所	ユーザーのPC	サーバーマシン
データ	一時的	永続的（DB保存）
見える？	ユーザーが見える	ユーザーから見えない

実習1: フロントエンドTODOの問題を確認する

1. 前回作成したフロントエンドTODOアプリを開く
2. TODOをいくつか追加する
3. **別のタブ** で同じURLを開く
4. 追加したTODOが表示されるか確認する

確認ポイント:

- 別タブではTODOが共有されていない
- ブラウザを閉じて再度開くとデータが消えている
- → これがフロントエンドだけの限界

セクション2

Python基礎 - 変数・型・リスト・辞書

なぜPythonを使うか

- **読みやすい** - 英語に近い文法
- **学びやすい** - 初心者向け言語として世界的に人気
- **Webバックエンド** に広く使われている
- **FastAPI** という高速なWebフレームワークがある

```
# JavaScriptとPythonの比較
# JavaScript
# let message = "Hello";
# console.log(message);

# Python
message = "Hello"
print(message)
```

変数と型

Pythonの基本的なデータ型:

```
# 文字列 (str)
name = "田中太郎"

# 整数 (int)
age = 20

# 小数 (float)
score = 85.5

# 真偽値 (bool)
is_student = True
```

ポイント: Pythonでは型を宣言しなくてよい（動的型付け）

型を確認する

```
name = "田中太郎"
age = 20
score = 85.5
is_student = True

print(type(name))       # <class 'str'>
print(type(age))         # <class 'int'>
print(type(score))       # <class 'float'>
print(type(is_student))  # <class 'bool'>
```

JavaScriptの `typeof` に相当するのが Pythonの `type()` 関数

リスト (list) - JavaScriptの配列に相当

```
# リストの作成
fruits = ["りんご", "バナナ", "みかん"]

# 要素へのアクセス（0始まり）
print(fruits[0])   # りんご
print(fruits[1])   # バナナ

# 要素の追加
fruits.append("ぶどう")
print(fruits)       # ["りんご", "バナナ", "みかん", "ぶどう"]

# 要素数
print(len(fruits))  # 4

# 要素の削除
fruits.remove("バナナ")
print(fruits)       # ["りんご", "みかん", "ぶどう"]
```

辞書 (dict) - JavaScriptのオブジェクトに相当

```
# 辞書の作成
todo = {
    "id": 1,
    "title": "レポートを書く",
    "done": False
}

# 値へのアクセス
print(todo["title"])      # レポートを書く

# 値の更新
todo["done"] = True
```

辞書 (dict)（続き）

```
# 新しいキーの追加
todo["priority"] = "高"

# 全てのキーと値を表示
print(todo)
# {'id': 1, 'title': 'レポートを書く', 'done': True, 'priority': '高'}
```

JavaScriptとPythonの対応表

JavaScript	Python	説明
`let x = 10`	`x = 10`	変数宣言
`[1, 2, 3]`	`[1, 2, 3]`	配列/リスト
`{key: "value"}`	`{"key": "value"}`	オブジェクト/辞書
`arr.push(x)`	`arr.append(x)`	配列に追加
`arr.length`	`len(arr)`	要素数
`console.log()`	`print()`	出力
`typeof x`	`type(x)`	型の確認
`true / false`	`True / False`	真偽値 (大文字に注意!)

実習2: ターミナルでPython入門

1. ターミナルを開き、Pythonインタラクティブモードを起動:

```
python
```

2. 以下を順番に試す:

```
name = "自分の名前"
print(name)
print(type(name))

todos = ["課題を出す", "買い物する", "掃除する"]
todos.append("洗濯する")
print(todos)

todo = {"id": 1, "title": "課題を出す", "done": False}
print(todo["title"])
```

3. `exit()` で終了

セクション3

Python基礎 - 関数・条件分岐・ループ

関数 (def)

```
# 関数の定義
def greet(name):
    return f"こんにちは、{name}さん！"

# 関数の呼び出し
message = greet("田中")
print(message)  # こんにちは、田中さん！
```

ポイント:

- `def 関数名(引数):` で定義
- **インデント（字下げ）** でブロックを表す（波括弧 `{}` は使わない）
- `return` で値を返す

f-string - 文字列の中に変数を埋め込む

```python
name = "田中"
age = 20

# f-string（Python 3.6以降）
message = f"私は{name}です。{age}歳です。"
print(message)  # 私は田中です。20歳です。

# 計算式も埋め込める
print(f"来年は{age + 1}歳です。")  # 来年は21歳です。

# JavaScriptのテンプレートリテラルに相当
# JavaScript: `私は${name}です。${age}歳です。`
# Python:     f"私は{name}です。{age}歳です。"
```

条件分岐 (if)

```
score = 75

if score >= 80:
    print("優秀です")
elif score >= 60:
    print("合格です")
else:
    print("もう少し頑張りましょう")

# 出力: 合格です
```

JavaScriptとの違い:

- `else if` → `elif`
- 波括弧 `{}` の代わりにインデント
- 条件の `()` は不要

ループ (for)

```python
# リストのループ
fruits = ["りんご", "バナナ", "みかん"]
for fruit in fruits:
    print(fruit)

# range を使ったループ
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4

# enumerate - インデックスと値を同時に取得
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
# 0: りんご
# 1: バナナ
# 2: みかん
```

辞書のリスト - TODOリストの表現

```
todos = [
    {"id": 1, "title": "レポートを書く", "done": False},
    {"id": 2, "title": "買い物に行く", "done": True},
    {"id": 3, "title": "掃除する", "done": False},
]

def show_todos(todo_list):
    for todo in todo_list:
        status = "[x]" if todo["done"] else "[ ]"
        print(f'{status} {todo["id"]}: {todo["title"]}')

show_todos(todos)
# [ ] 1: レポートを書く
# [x] 2: 買い物に行く
# [ ] 3: 掃除する
```

実習3: Pythonスクリプトの作成

1. `session05/exercise/` ディレクトリに移動
2. `basics.py` を作成して以下を書く:

```
todos = [
    {"id": 1, "title": "課題を出す", "done": False},
    {"id": 2, "title": "買い物する", "done": True},
    {"id": 3, "title": "自分のTODO", "done": False},
]

def show_todos(todo_list):
    for todo in todo_list:
        status = "[x]" if todo["done"] else "[ ]"
        print(f'{status} {todo["id"]}: {todo["title"]}')

show_todos(todos)
```

3. ターミナルで実行: `python basics.py`

セクション4

FastAPIとは - Hello World

Webフレームワークとは

Webフレームワーク = Webアプリケーションを作るための「骨組み」

- HTTPリクエストの受け取り
- URLとプログラムの紐付け（ルーティング）
- レスポンスの返却

これらを自分で一から作らなくてよくなる

FastAPIの特徴

特徴	説明
高速	Pythonの中で最速レベル
簡単	少ないコードでAPIが書ける
自動ドキュメント	API仕様書を自動生成
型ヒント対応	エディタの補完が効く

FastAPIの最小構成

```python
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}
```

コード解説:

1. `from fastapi import FastAPI` - FastAPIをインポート
2. `app = FastAPI()` - アプリケーション（サーバー）を作成
3. `@app.get("/")` - 「GET / にアクセスが来たらこの関数を実行」
4. `return {"message": ...}` - 辞書を返す → 自動でJSONに変換

サーバーの起動方法

起動コマンド:

```
$ python main.py
```

実習4: FastAPI Hello World

1. ディレクトリの移動:

```
cd session05/exercise
```

2. `main.py` を開き、最初のエンドポイントを確認
3. サーバーを起動:

```
python main.py
```

4. ブラウザで `http://localhost:8000` にアクセス
5. `{"message": "Hello, World!"}` が表示されることを確認
6. 自動ドキュメント: `http://localhost:8000/docs` にもアクセスしてみよう

セクション5

ルーティングとエンドポイント

ルーティングとは

ルーティング = URLパスと関数の対応付け

```
@app.get("/")              # GET http://localhost:8000/
def read_root():
    return {"message": "Hello"}

@app.get("/about")         # GET http://localhost:8000/about
def about():
    return {"page": "About"}

@app.get("/contact")       # GET http://localhost:8000/contact
def contact():
    return {"page": "Contact"}
```

URLが異なれば、異なる関数が実行される

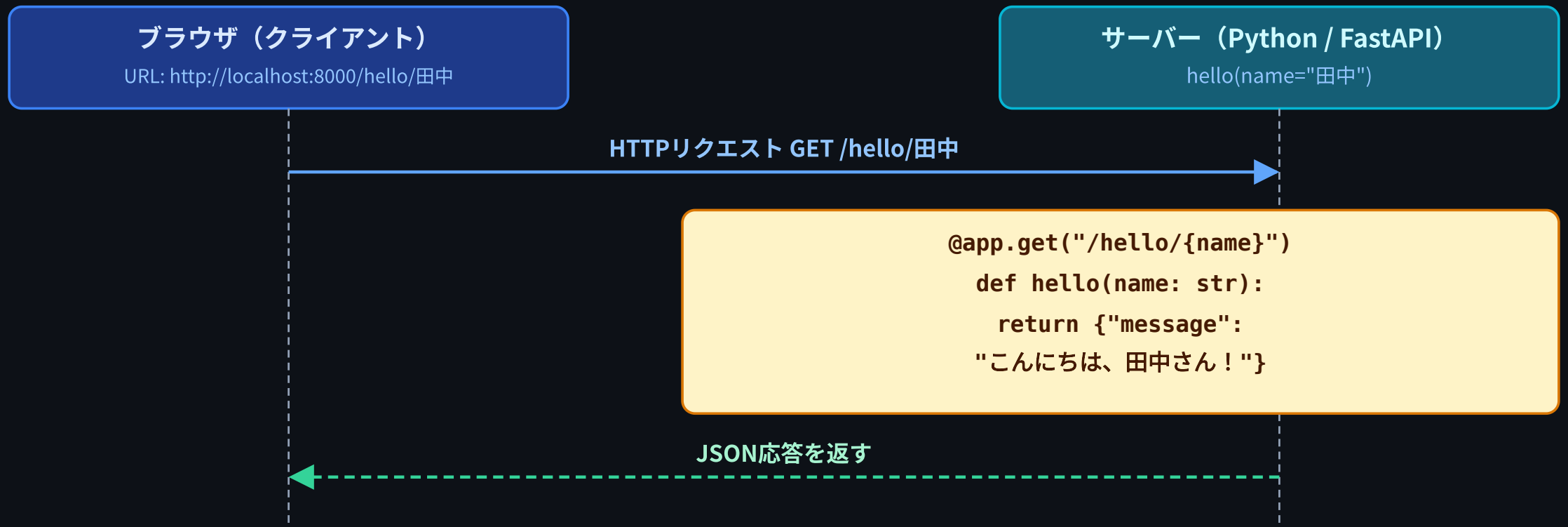
パスパラメータ - URLに変数を埋め込む

```python
@app.get("/hello/{name}")
def hello(name: str):
    return {"message": f"こんにちは、{name}さん！"}
```

```
アクセス例:
GET /hello/田中   →  {"message": "こんにちは、田中さん！"}
GET /hello/鈴木   →  {"message": "こんにちは、鈴木さん！"}

URLの一部が変数になる:
/hello/{name}
       ~~~~~~
       この部分が関数の引数 name に入る
```

コードがサーバーで実行されることを理解する



HTTPメソッドの種類（プレビュー）

HTTPメソッド	用途	FastAPIデコレータ
GET	データの取得	`@app.get()`
POST	データの作成	`@app.post()`
PUT	データの更新	`@app.put()`
DELETE	データの削除	`@app.delete()`

今回は **GET** のみ使用。次回以降で他のメソッドも学ぶ。

実習5: パスパラメータの実装

1. `main.py` に `/hello/{name}` エンドポイントを追加:

```
@app.get("/hello/{name}")
def hello(name: str):
    return {"message": f"こんにちは、{name}さん！"}
```

2. ブラウザで以下のURLにアクセスして確認:
   - `http://localhost:8000/hello/自分の名前`
   - `http://localhost:8000/hello/FastAPI`
3. **確認ポイント:**
   - URLを変えると返ってくるデータが変わる
   - Pythonのコードがサーバー上で実行されている
   - ブラウザにはその結果だけが届く

セクション6

JSONとデータ交換

JSONとは

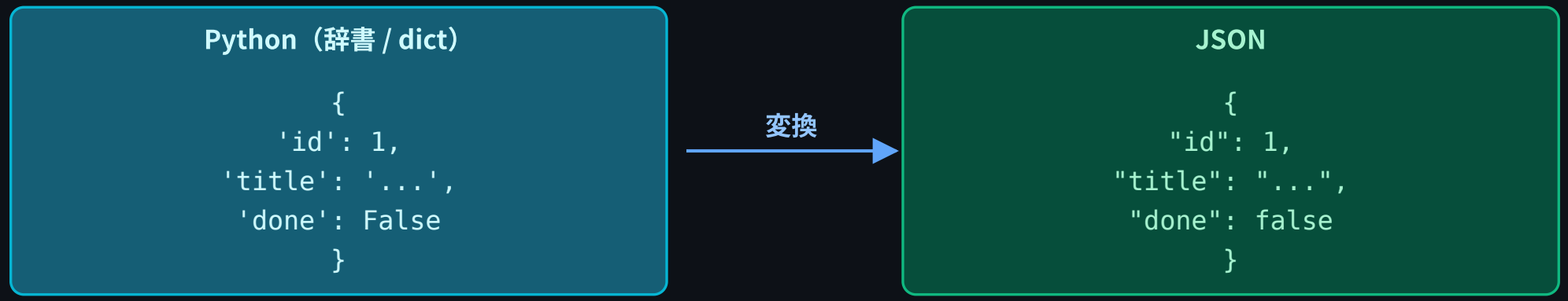
JSON (JavaScript Object Notation) = データ交換用のフォーマット

```
{
    "id": 1,
    "title": "レポートを書く",
    "done": false
}
```

特徴:

- テキスト形式なので人間も読める
- 多くのプログラミング言語でも扱える
- Web APIの標準的なデータ形式

PythonのdictとJSONの対応



ほぼ同じ！ 違いは:

- Python: `True/False` → JSON: `true/false`
- Python: `None` → JSON: `null`

FastAPIは **Pythonの辞書を自動でJSONに変換** してくれる

TODOリストをAPIで返す

```
todos = [
    {"id": 1, "title": "レポートを書く", "done": False},
    {"id": 2, "title": "買い物に行く", "done": True},
    {"id": 3, "title": "掃除する", "done": False},
]

@app.get("/todos")
def get_todos():
    return todos
```

`GET /todos` にアクセスすると:

```
[
  {"id": 1, "title": "レポートを書く", "done": false},
  {"id": 2, "title": "買い物に行く", "done": true},
  {"id": 3, "title": "掃除する", "done": false}
]
```

フロントエンドとバックエンドの連携イメージ

- フロントエンドは「表示」を担当
- バックエンドは「データ管理」を担当
- JSONでデータをやり取り

自動ドキュメント (Swagger UI)

FastAPIは自動で **API仕様書** を生成する

`http://localhost:8000/docs` にアクセスすると:

- 全エンドポイントの一覧
- リクエスト/レスポンスの形式
- **その場でAPIを試せる** "Try it out" ボタン

これは開発中にとても便利。フロントエンド開発者がバックエンドのAPIを確認する際にも使われる。

実習6: TODO APIの作成

1. `main.py` にTODOデータとエンドポイントを追加:

```
todos = [
    {"id": 1, "title": "レポートを書く", "done": False},
    {"id": 2, "title": "買い物に行く", "done": True},
    {"id": 3, "title": "自分のTODOを追加", "done": False},
]

@app.get("/todos")
def get_todos():
    return todos
```

2. ブラウザで `http://localhost:8000/todos` にアクセス
3. JSON形式でTODOリストが返ってくることを確認
4. `http://localhost:8000/docs` でSwagger UIも確認

実習6 (続き): Git commit & push

5. 動作確認ができたら、Git でコミット:

```
cd /workspaces/web-app-intro-class-2026
git add session05/
git commit -m "第5回: Python & FastAPI入門の実習コードを追加"
git push
```

今日のまとめ

【学んだこと】

1. バックエンドが必要な理由
   → データ永続化、複数ユーザー対応、セキュリティ
2. Python基礎
   → 変数、型、リスト、辞書、関数、ループ
3. FastAPI
   → Webフレームワーク、サーバー起動、ルーティング
4. JSON
   → データ交換フォーマット、Pythonの辞書と対応

【コードの実行場所】

ブラウザ (JavaScript) → サーバー (Python)
└── 今回ここが変わった！

次回予告

第6回: データベース（SQLite）& REST API

- データベースでデータを永続化
- REST APIの設計と実装（CRUD操作）
- Swagger UIでのテスト

今回作った GET /todos を発展させ、データベースと連携した本格的なAPIを構築する

提出物

実習で穴埋め・実装を完了させたファイルをフォームから提出してください:

1. `main.py` のGitHubのURL
    - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session05/exercise/main.py`
2. `basics.py` のGitHubのURL
    - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session05/exercise/basics.py`